GRADE 6 · CODING CLUB · LESSON 4

Pet Shelter API

FastAPI + Pydantic together: add, list and find pets

Our API will say "no thanks" to bad data — all by itself.

TODAY

1. GET vs POST
2. A Pet model
3. POST /pets
4. Try it in /docs
5. When data is wrong
6. Finding one pet + 404
7. Build + showcase

Reading vs adding

	GET	POST
Means	"Give me information"	"Here is new information — please save it"
Restaurant	"Can I see the menu?"	"Here's my order"
Data travels in…	the address (path and query)	the **body** of the request, as **JSON**
Pet Shelter example	`GET /pets` — list all pets	`POST /pets` — add a new pet

TALKING POINTS

The new idea today is the request body: when we POST, we send a whole JSON object, not just a value in the address. That JSON needs checking — the perfect job for a Pydantic model.

The Pet Shelter API


```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI(title="Pet Shelter API",
              description="Our class pet shelter!")
pets = []                    # our pets live in this list

class Pet(BaseModel):
    name: str = Field(min_length=1, max_length=20)
    species: str
    age: int = Field(ge=0, le=30)

@app.get("/pets")
def list_pets():
    return pets

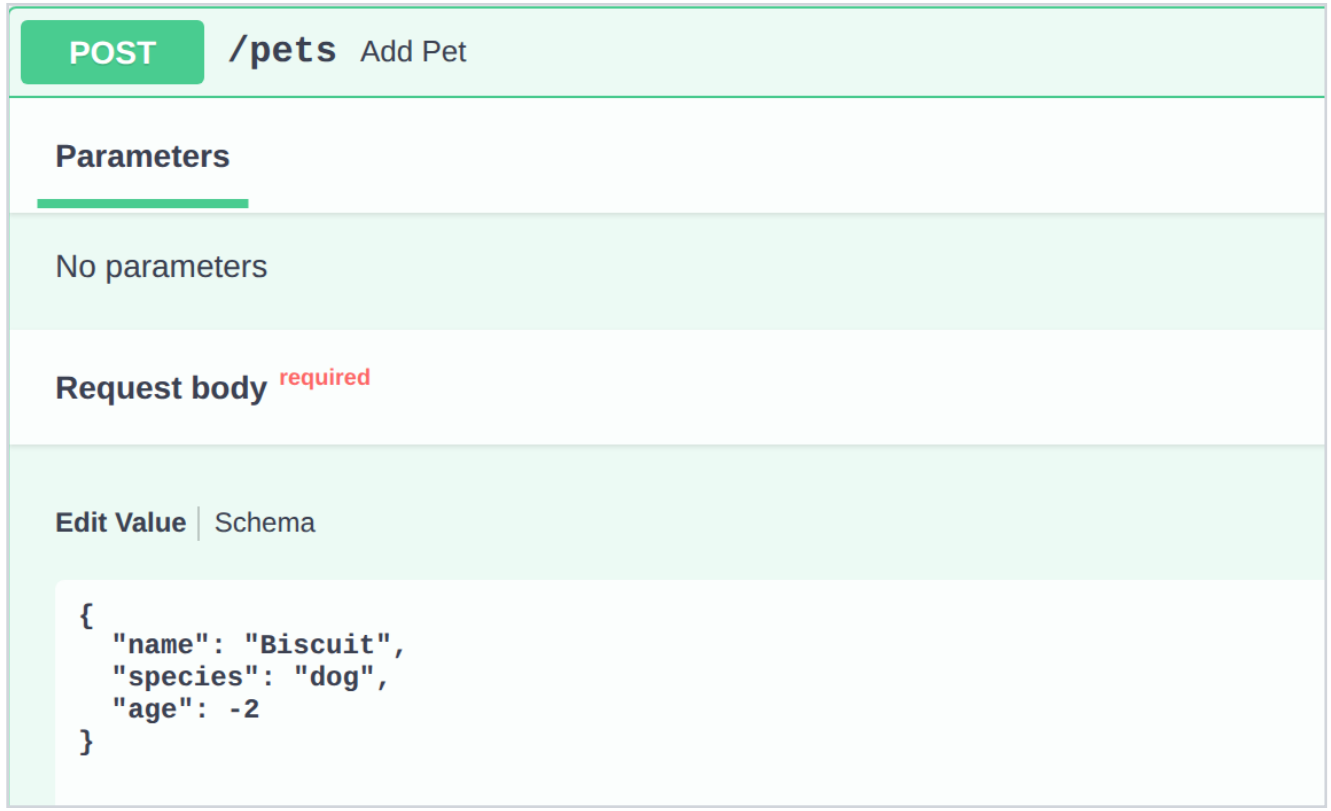
@app.post("/pets")
def add_pet(pet: Pet):       # FastAPI checks it with Pydantic!
    pets.append(pet)
    return {"added": pet, "total": len(pets)}
```


TALKING POINTS

Key line: add_pet(pet: Pet). Because the type is a Pydantic model, FastAPI knows to read JSON from the request body, check it with the model's rules, and give our function a ready-made Pet. If the data is bad, our function never even runs. Note: the list resets when the server restarts — real apps use a database.

Try it in /docs: add a pet

Open `/docs` → **POST /pets** → **Try it out** → type the JSON → **Execute**



FastAPI already shows an **example** JSON body, built from your Pet model

Here someone typed **"age": -2**…

What do you think will happen?

TALKING POINTS

Do this live first with good data (Biscuit, dog, 3) and show the 200 response with total 1. Then add a second pet and GET /pets. Then try the -2 age shown here and ask for predictions before pressing Execute.

When the data is wrong: error 422

The API says **no** — and explains why. We didn't write any of this checking code!

Code	Details
422	Error: Unprocessable Entity

Response body


```
{
  "detail": [
    {
      "type": "greater_than_equal",
      "loc": [
        "body",
        "age"
      ],
      "msg": "Input should be greater than or equal to 0",
      "input": -2,
      "ctx": {
        "ge": 0
      }
    }
  ]
}
```


422 = "I understood you, but the data breaks the rules"

loc: where — body → age

msg: what's wrong

input: what you sent

TALKING POINTS

Compare with the Checker in the Human API game: it rejected bad orders and gave a reason. The same Pydantic messages from Lesson 2 now come back as JSON. Let kids try to break the API in other ways: empty name, missing species, age 'old', a 25-letter name.

Status codes: the API's mood

Code	Meaning	When
200	OK!	Everything worked
404	Not found	You asked for something that isn't there (pet number 99)
422	Data breaks the rules	Pydantic said no
500	The server made a mistake	A bug in our code crashed

TALKING POINTS

Fun fact: 404 is the famous 'page not found' number kids may have seen on websites. Codes starting with 2 are good, 4 means the request has a problem, 5 means the server has a problem.

Finding one pet — and saying "not found" politely

Remember `/facts/99` crashing with error 500? This is the fix.

```
from fastapi import HTTPException

@app.get("/pets/{pet_id}")
def get_pet(pet_id: int):
    if pet_id < 0 or pet_id >= len(pets):
        raise HTTPException(status_code=404,
                            detail="No pet with that number")
    return pets[pet_id]
```

`/pets/0` → the first pet

`/pets/99` → **404**
`{"detail": "No pet with that number"}`

TALKING POINTS

raise HTTPException stops the function and sends a friendly error with the right status code. This is good manners for an API: a clear message instead of a crash.

Build your shelter (25 minutes)

- Type in the Pet Shelter API and run it with `fastapi dev main.py`
- In **/docs**: add 3 pets, list them, find one by number
- **Level up** — choose at least one:
  - Add a rule: species must be 3 to 15 letters
  - Add an optional field `favourite_toy: str | None = None`
  - Add `GET /pets/count` that returns how many pets there are
  - Add `GET /pets/species/{species}` that returns only cats, dogs…
  - Change the title and description so **/docs** shows your shelter's name

TALKING POINTS

Tip: GET /pets/count must be written ABOVE GET /pets/{pet_id} in the file, or FastAPI will try to treat 'count' as a pet number and give a 422 — a great debugging moment for strong students. For species filtering: return [p for p in pets if p.species == species].

Showcase + quiz

- **Showcase:** show your /docs page · add a pet live · then try to break it with bad data and explain the error
- 1. How does FastAPI know to check the pet data?
- 2. What does error **422** mean? And **404**?
- 3. Where does the data travel in a **POST** request?
- 4. Why is it a good idea for an API to check data before saving it?
- You built a real web API — the same tools professional programmers use!

TALKING POINTS

Answers: 1) The function input has a Pydantic model type (pet: Pet), so FastAPI validates the JSON body with it. 2) 422: the data breaks the rules; 404: not found. 3) In the request body, as JSON. 4) To stop bugs, crashes and nonsense data (and keep it safe).